Generative_Models

Generative models

1. "What I cannot create I do not understand" - Richard Feynman. Generative modeling is based on the contrapositive of this "what I understand I should be able to create".
2. Given observed samples x from a distribution of interest, the goal of generative model is to learn to model its true distribution $p(x)$.
3. **Key idea 1**: data as vectors $z \in \mathrm{R}^d$ (tensors, vectorized). e.g. $vec(\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}) = [1 \quad 2 \quad | \quad 3 \quad 4]$
4. **Key idea 2**: generation as sampling, generation is sampling. Generating an object z is modeled as sampling from the data distribution $z \sim p_{data}$. Meaning select z according to the p_{data} (but p_{data} is unknown) Data distribution is a prob distribution over data space R^d. Described by probability density function p_{data}.
5. A generative model is a machine learning model that allows us to generate samples from p_{data}. In machine learning we require data to train models. In generative modeling, we usually assume access to a finite number of examples sampled independently from p_{data}, which together serve as a proxy for the true distribution.
6. **Key idea 3**: p_{data} is approximated by an empirical distribution, formed by a dataset, a random sample $z_1, z_2, \ldots, z_N \sim p_{data}$.
7. 

8. **Key idea 4**: Guided generation involves sampling from $z \sim p_{data}(\cdot|y)$, where y is a conditioning variable.
    1. Guided/conditioned generation: y: "a dog running down a hill covered with snow with mountains in the background". We can paraphrase this as sampling from a conditional distribution.
    2. The guided generative modeling task typically involves learning to condition on an arbitrary, rather than fixed, choice of y. It turns out that techniques for unconditional generation are readily generalized to the conditional case.
9. Generative modelling
    1. Z is some standard random variable.
    2. f_θ is a neural network with parameters θ.
    3. X is a parametric RV whose distribution depends on $f_\theta(Z)$.
    4. How is the training? **Training any probability model is always the same, its just maximum likelihood estimation**. We want to pick the neural network parameters so as to maximize the log likelihood of the dataset. We'll assume that our dataset consists of N independent samples x_1 to x_N.
    5. Training: MLE: we want θ to maximize log likelihood of dataset.
        1. $$\begin{aligned}\log Pr(x_1, \ldots, x_n; \theta) &= \sum_i \log Pr_x(x_i; \theta) \\ &= \sum_i \log \int_z Pr_x(x_i|Z = z; \theta) Pr_z(z) \mathrm{dz}\end{aligned}$$
        2.

Autoencoder

1. How NOT to train an autoencoder
    1. Define a reconstruction loss, call it $L(\tilde{x}, x)$.
    2. Train enc. and dec. to minimize loss on the training set.
    3. Evaluate loss on the holdout dataset.
        1. This won't work because, the trained autoencoder can reconstruct the dataset but it does not extract useful features for the input, so there is no guarantee that semi-supervised learning would work, and random generation won't work either we're just spitting random variable z the our thought experiment autoencoder will only produce noise not an interesting output.
        2. So we would need to think much harder on how to train an autoencoder.
        3. The right way to train an autoencoder is discovered by two independent researchers in 2014. The answer is probabilistic modelling, specifically generative modelling.

2. Variational autoencoder (VAE)
    1. We need to do two things
        1. Maximize the likelihood of data X -> maximize $\log p_\theta(x)$
        2. Make sure that the Z we get from encoding X can actually be decoded into the same X -> minimize the "difference" between $q_\phi(z|x)$ and $p_\theta(z|x)$
        3. what we can design: $p_\theta(x), q_\phi(z|x), p_\theta(x|z)$
        4. what we don't have: $p_\theta(x), p_\theta(z|x)$
        5. what we want: $p_\theta(x), q_\phi(z|x), p_\theta(z|x)$
        6. Vae's evidence lower bound (ELBO)
            1. we are trying to
                1. Maximize the likelihood of data X -> maximize $\log p_\theta(x)$
                2. Make sure that the Z we get from encoding X can actually be decoded into the same X -> minimize the "difference" between $q_\phi(z|x)$ and $p_\theta(z|x)$
                3. $\arg\max_{\phi,\theta}\left(\log p_\theta(x) - D_{KL}(q_\phi(z|x)||p_\theta(z|x))\right)$
        7. Two ways to derive ELBO
            1. $$\begin{aligned}\log p_\theta(x) &= \log \int p_\theta(x,z)dz \\ &= \log \int \frac{q_\phi(z|x)}{q_\phi(z|x)} p_\theta(x,z)dz \\ &= \log E_{q_\phi(z|x)}[\frac{p_\theta(x,z)}{q_\phi(z|x)}] \\ &\geq E_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z|x)}\right]\end{aligned}$$
            2. Jensen's inequality:
                1. for convex function: essentially is saying "the expectation of the function is geq to the function applied to the expectation".
                2. for concave function: the sign flips. Log is a concave function.
            3.

Diffusion models

1. Connection to VAEs
    1. Diffusion models can be considered as special case of hierarchical VAEs.
    2. However, in diffusion models:
        1. The *encoder is fixed*.
        2. The latent variables have the same dimension as the data.
        3. The denoising model is shared across different timestep.
        4. The model is trained with some reweighting of the variational bound.
2. DDPM is an image-generation model
    1. DDPM is inspired by indirect sampling technique like **Gibbs sampling**, to sample a uniform distribution and feed it into a Markov Chain.
    2. Generating images=Sampling images from a simple prior.
    3. Modeling the distrib of the data X of interest.
    4. Computing the prob of data p(x), x is an image.
    5. In contrast to discriminative models that models p(y|x).
    6. DDPM does not learn the variance, just the mean of the noise ϵ.
    7. ELBO of DDPM
        1. $$ELBO = \mathrm{E}_q[\log(x_0|x_1) - D_{KL}(q(\mathbf{x}_T|x_0)||p(x_T)) - \sum_{T>1} D_{KL}(q(x_{t-1}|x_t,x_0)||p_\theta(x_{t-1}|x_t)]$$
        2. *Reconstruction term*: $\mathrm{E}_q[\log p_\theta(x_0|x_1)]$ or as $L_0 = -E_q[\log p_\theta(x_0|x_1)]$. The reconstruction term is derived from the expectation over the conditional distribution $q_\phi(x_1|x_0)$. This term predicts the log probability of the original data sample $\mathbf{x}_0$ given the first-step latent $\mathbf{x}_1$. This is similar to standard VAE, where the model learns to regenerate the original input from its latent representation, enhancing the model's ability to capture and reconstruct the input data accurately.

3. *Prior matching term*: $-E_q[D_{KL}(q(\mathbf{x}_T|x_0)||p(x_T))]$ (minus sign because we want to minimize) or as $L_T = E_q[D_{KL}(q(\mathbf{x}_T|x_0)||p(x_T))]$. This term involves the KL divergence between the *final* latent distribution approximates a Gaussian, this term often evaluates to zero. This simplification reflects the model's adherence to the prior distribution without requiring trainable parameters.
4. *Denoising matching (consistency) term*: $-E_q[\sum_{T>1} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)]$ or as $L_{t-1} = E_q[\sum_{T>1} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)]$. The consistency term measures the fidelity of the denoising transitions. It ensures that the model's denoising capability, from a noisier to a cleaner state, matches the theoretical ground-truth denoising transition defined by $q(x_t|x_t, x_0)$. This term is minimized when the model's predicted denoising steps closely align with these ground-truth transitions.

8. ELBO continued: computational details
    1. We need the explicit form of $q(\mathbf{x}_t, \mathbf{x}_0)$ (for forward process) and $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ (as ground truth for reverse process) for sampling.
    2. We present the result here:

    $$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}),$$

    or equivalently

    $$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_0$$

    , where $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$ and $\epsilon_0 \sim \mathcal{N}(0, \mathbf{I})$.

    3. $$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}|\mu_q(\mathbf{x}_t, \mathbf{x}_0), \mathbf{\Sigma}_q(t)),$$

    where

    $$\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_t}\mathbf{x}_0$$

    , and

    $$\mathbf{\Sigma}_q(t) = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{I} =^{def} \sigma_q^2(t)\mathbf{I}$$

    .
    4. First way of modeling: Model to directly predict mean
        1. From the ELBO, we know that we have to compute the KL divergence term. p_θ is what we can set for training, and we know that $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is Gaussian, so for convenience we formulate p_θ as a Gaussian, with the same variance as $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and a learnable mean, i.e. *what we want to learn*:

        $$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}|\mu_\theta(\mathbf{x}_t, t), \sigma_q^2(t)\mathbf{I})$$

        where $\mu_\theta(\mathbf{x}_t, t)$ is a neural network parametrized by θ.
        2. Now we have $D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) = \frac{1}{2\sigma_q^2(t)}||\mu_q(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2$.
        3. To match the form of mean in ground truth,

        $$\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_t}\mathbf{x}_0$$

        we set

        $$\mu_\theta(\mathbf{x}_t) =^{def} \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_t}\hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)$$

        , where $\hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)$ is another neural network (still parametrized by θ) that predicts the clean image $\mathbf{x}_0$ from the noisy image $\mathbf{x}_t$ and time t.
    5. Second way of modeling: Model to predict noise
        1. Note that we set our neural network ($\hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)$) for predicting the image. We can actually learn to predict the noise. To see that, from $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\epsilon_0$ we can obtain x_0 using the *reparametrization trick*, $\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{(1-\bar{\alpha})}\epsilon_0}{\sqrt{\bar{\alpha}_t}}$ and put it in $\mu_q(\mathbf{x}_t, \mathbf{x}_0)$, and then get
        2. $\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}\sqrt{\alpha_t}}\epsilon_0$.
        3. So similarly we can design our mean estimator μ_θ as
        4. $\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}\sqrt{\alpha_t}}\hat{\epsilon}_\theta(\mathbf{x}_t, t)$.

9. Training:
    1. Forward process (encoder q, fixed): from x_0 to x_T
        1. we can fix the forward process to make learning easier.
        2. $\alpha =^{def} 1-\beta_t$ and $\bar{\alpha}_t =^{def} \prod_{s=1}^{t} \alpha_s$
        3. $q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})$
    2. Reverse process (denoising, decoder p_θ, learned): from x_T to x_0
        1. $q(x_{t-1}|x_t)$ is intractable, but $q(x_{t-1}|x_0, x_t)$ is not.
        2. using Bayes rule: $q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)}$
        3. As we already know that $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = q(x_t|x_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, (1-\alpha_t)\mathbf{I})$ from our assumption regarding encoder transitions, what remains is deriving for the forms of $q(\mathbf{x}_{t-1}|\mathbf{x}_0)$ and $q(\mathbf{x}_t|\mathbf{x}_0)$.
        4. $q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}(x_t, x_0), \tilde{\beta}_t\mathbf{I})$
        5. where $\tilde{\mu}_t(\mathbf{x}_t. \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t$ and $\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$
        6. Similarly we can also fix the variance of the reverse process, and learn only the mean
        7. $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2\mathbf{I})$
        8. $L_{t-1} = E_q[\frac{1}{2\sigma_t^2}||\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2 + C$
10. Summary:
    1. Forward process: Diffusion to get the input.
        1. Sample an image x from the training set.
        2. Sample noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$.
        3. Sample a variance $\beta \in (0, 1)$.
        4. Get the input $x_t = \sqrt{1-\beta}x + \sqrt{\beta}\epsilon$.
        5. reparametrization trick: $x = mean + std\_dev * Z$ to sample from normal distribution. Z is drawn from a standard normal distribution.
        6. Note: reparametrization trick is a useful mental step to do backprop through a sampling process. While the gradient of an individual sample indeed does not exist or isn't meaningful, it does exist in expectation, and the expectation is all you need to backprop. Papers that refer this as a trick may come from the years 2012-2014.
    2. Backward process: Denoising to get the output
        1. Feed the input into the network.
        2. Network: U-Net f_θ.
        3. Predict the noise $f_\theta(x_t, t)$.
        4. Loss is MSE.
        5. input to model: $x_t = \sqrt{1-\beta}x + \sqrt{\beta}\epsilon$
        6. output: $\tilde{\epsilon}$, $\tilde{x} = \frac{x_t - \sqrt{\beta}\tilde{\epsilon}}{\sqrt{1-\beta}}$, denoising: noisy image x_t minus the noise $\sqrt{\beta}\epsilon$
11. Inference: Iterative denoising

References:

1. Understanding Diffusion Models: A Unified Perspective, Calvin Luo.
2. CS182 Deep Learning, UC Berkeley, Sergey Levine, 2021.
3. CS330 Variational Inference and Generative models, Chelsea Finn, Stanford, 2022.
4. https://jiegroup-genai.readthedocs-hosted.com/en/latest/diffusion/index.html#denoising-diffusion-probabilisitc-model-ddpm, DDPM, Umass Amherst.
5. DDPM in Keras, https://keras.io/examples/generative/ddpm/
6. The Principles of Diffusion Models, Chieh-Hsin Lai et al.